

Implémentations Multiples du Morpion avec IA

Nom : Yehdih Mohamed

Matricule : C20854

Nom : Elbou Aly Sidina

Matricule : C20851 2 mai

Nom : Mohamed Elmoktar Sidaty Hadoueni

Matricule : C30888

2025

Table des matières

1	Introduction	2
1.1	Contexte du Projet	2
1.2	Objectifs du Projet	2
1.2.1	Objectifs Techniques	2
1.2.2	Objectifs Pédagogiques	2
2	Architecture Comparative	2
2.1	Analyse des Architectures	2
2.2	Version Streamlit	2
2.3	Version Tkinter	3
3	Implémentation de l'IA	3
3.1	Algorithme Minimax Commun	3
3.1.1	Principe Mathématique	3
3.2	Gestion des Niveaux de Difficulté	4
4	Version Streamlit	5
4.1	Caractéristiques Uniques	5
4.2	Structure du Code	5
4.3	Extrait de Code Détaillé	5
5	Version Tkinter	6
5.1	Caractéristiques Uniques	6
5.2	Structure du Code	6

1 Introduction

1.1 Contexte du Projet

Ce projet présente deux implémentations distinctes d'un jeu de Morpion (Tic-Tac-Toe) avec intelligence artificielle, développées dans le cadre d'un cours avancé d'algorithmique et de programmation. Le morpion, bien que simple en apparence, représente un excellent cas d'étude pour l'implémentation d'algorithmes d'intelligence artificielle, notamment l'algorithme Minimax. Les deux versions développées offrent des approches complémentaires :

- Version 1 : Interface Streamlit (web) - Solution moderne adaptée au déploiement en ligne
- Version 2 : Interface Tkinter (desktop) - Application native pour une utilisation locale

1.2 Objectifs du Projet

Les objectifs principaux de ce projet étaient :

1.2.1 Objectifs Techniques

- Implémentation rigoureuse de l'algorithme Minimax avec élagage alpha-bêta
- Gestion des différents niveaux de difficulté (Facile, Moyen, Difficile)
- Conception d'interfaces utilisateur intuitives et ergonomiques
- Mécanisme de jeu tour par tour avec gestion d'état robuste

1.2.2 Objectifs Pédagogiques

- Compréhension approfondie des algorithmes de jeu
- Comparaison des paradigmes web vs desktop
- Gestion de projet collaboratif avec Git
- Documentation technique complète

2 Architecture Comparative

2.1 Analyse des Architectures

2.2 Version Streamlit

- **Framework web** : Basé sur Python, Streamlit permet de créer des applications web rapidement

- **Architecture client-serveur** : Le navigateur communique avec un serveur Python

- **Gestion d'état** : Utilisation intensive de `session_state` pour préserver l'état entre les interactions

- **Déploiement** : Facilité de déploiement sur des plateformes cloud comme Streamlit Sharing, Heroku ou AWS

- **Avantages** : Accessibilité depuis n'importe quel appareil connecté, pas d'installation requise

- **Inconvénients** : Latence réseau, dépendance à une connexion internet

2.3 Version Tkinter

- **Interface desktop** : Application native utilisant les widgets du système
- **Architecture** : Modèle monolithique avec toutes les fonctionnalités intégrées
- **Gestion d'état** : Variables de classe et callbacks pour gérer l'état du jeu
- **Exécution** : Simple fichier exécutable ou script Python local
- **Avantages** : Temps de réponse instantané, fonctionnement hors-ligne
- **Inconvénients** : Déploiement plus complexe, interface moins moderne

3 Implémentation de l'IA

3.1 Algorithme Minimax Commun

Les deux versions utilisent le même cœur algorithmique basé sur Minimax, un algorithme récursif qui explore l'arbre des possibles pour déterminer le meilleur coup. L'implémentation comprend également une optimisation par élagage alpha-bêta pour réduire l'espace de recherche.

3.1.1 Principe Mathématique

L'algorithme Minimax repose sur la théorie des jeux à somme nulle et la notion d'équilibre de Nash. Pour un jeu à deux joueurs où un gain pour un joueur correspond à une perte pour l'autre, Minimax trouve la stratégie optimale.

La fonction d'évaluation est définie comme :

$$\text{eval}(b) = \begin{cases} +10 & \text{si l'IA gagne} \\ -10 & \text{si le joueur humain gagne} \\ 0 & \text{en cas de match nul} \end{cases}$$

De plus, la profondeur dans l'arbre influence le score final pour encourager des décisions rapides :

$$\text{score}_{\text{ajusté}} = \text{eval}(b) - \text{profondeur}$$

Algorithm 1 Algorithme Minimax avec élagage alpha-bêta

Require: Plateau, profondeur, estMaximisant, alpha, beta

```
1: if Partie terminée ou profondeur maximale atteinte then
2:   return score d'évaluation
3: end if
4: if estMaximisant then
5:   meilleur =  $-\infty$ 
6:   for chaque coup possible do
7:     effectuer le coup temporaire
8:     score = minimax(plateau, profondeur+1, False, alpha, beta)
9:     annuler le coup
10:    meilleur = max(meilleur, score)
11:    alpha = max(alpha, meilleur)
12:    if beta  $\leq$  alpha then
13:      break (élagage)
14:    end if
15:  end for
16:  return meilleur
17: else
18:   meilleur =  $+\infty$ 
19:   for chaque coup possible do
20:     effectuer le coup temporaire
21:     score = minimax(plateau, profondeur+1, True, alpha, beta)
22:     annuler le coup
23:     meilleur = min(meilleur, score)
24:     beta = min(beta, meilleur)
25:     if beta  $\leq$  alpha then
26:       break (élagage)
27:     end if
28:   end for
29:   return meilleur
30: end if
```

3.2 Gestion des Niveaux de Difficulté

Les trois niveaux de difficulté sont implémentés comme suit :

TABLE 1 – Paramètres des niveaux de difficulté

Niveau	Profondeur Maximale	Élagage Alpha-Bêta
Facile	2	Non
Moyen	4	Oui
Difficile	8	Oui

4 Version Streamlit

4.1 Caractéristiques Uniques

- **Trois niveaux de difficulté** : Paramétrables dynamiquement via un sélecteur
- **Historique des parties** : Stockage des résultats dans `session_state`
- **Scores persistants** : Conservation des statistiques entre les rafraîchissements
- **Interface responsive** : Adaptée aux mobiles et tablettes
- **Thèmes personnalisables** : Choix entre light/dark mode

4.2 Structure du Code

La version Streamlit est organisée en plusieurs modules :

- `game_logic.py` : Contient les règles du jeu et l'algorithme Minimax
- `streamlit_app.py` : Gère l'interface utilisateur et les interactions
- `utils.py` : Fonctions utilitaires pour le rendu et la gestion d'état

4.3 Extrait de Code Détaillé

Listing 1 – Implémentation complète de Minimax dans Streamlit

```
def minimax(board, depth, is_maximizing, alpha=-float('inf'), beta=
    ↪ float('inf')):
    """ Implémentation de l'algorithme Minimax avec lagage alpha-
        ↪ beta
    Args:
        board (list): Matrice 3x3 représentant le plateau
        depth (int): Profondeur actuelle de la recherche
        is_maximizing (bool): True si c'est le tour de l'IA
        alpha (float): Valeur alpha pour l' lagage
        beta (float): Valeur beta pour l' lagage
    Returns:
        int: Score du plateau (-10 à +10)
    """
    winner = check_winner(board)
    # Conditions d'arrêt
    if winner == AI_MARK:
        return 10 - depth # Pénalise les victoires lointaines
    if winner == HUMAN_MARK:
        return depth - 10 # Favorise les défaites lointaines
    if is_full(board):
        return 0
    if is_maximizing:
        best_score = -float('inf')
        for move in get_available_moves(board):
            i, j = move
            board[i][j] = AI_MARK
            score = minimax(board, depth+1, False, alpha, beta)
            board[i][j] = EMPTY
            best_score = max(score, best_score)
```

```

        alpha = max(alpha, best_score)
        if beta <= alpha:
            break # lagage beta
    return best_score
else:
    best_score = float('inf')
    for move in get_available_moves(board):
        i, j = move
        board[i][j] = HUMAN_MARK
        score = minimax(board, depth+1, True, alpha, beta)
        board[i][j] = EMPTY
        best_score = min(score, best_score)
        beta = min(beta, best_score)
        if beta <= alpha:
            break # lagage alpha
    return best_score

```

5 Version Tkinter

5.1 Caractéristiques Uniques

- **Interface native** : Meilleure intégration avec le système d'exploitation
- **Temps de réponse** : Réactivité immédiate grâce à l'exécution locale
- **Indépendance** : Fonctionne sans connexion internet
- **Personnalisation** : Contrôle total sur les widgets et le rendu

5.2 Structure du Code

L'application Tkinter suit une architecture MVC simplifiée :

- **Modèle** : Gère la logique du jeu et l'état
- **Vue** : Crée et met à jour l'interface
- **Contrôleur** : Gère les interactions utilisateur